SECURITY



Advancing Caliptra Security Through Open-Source DV Infrastructure

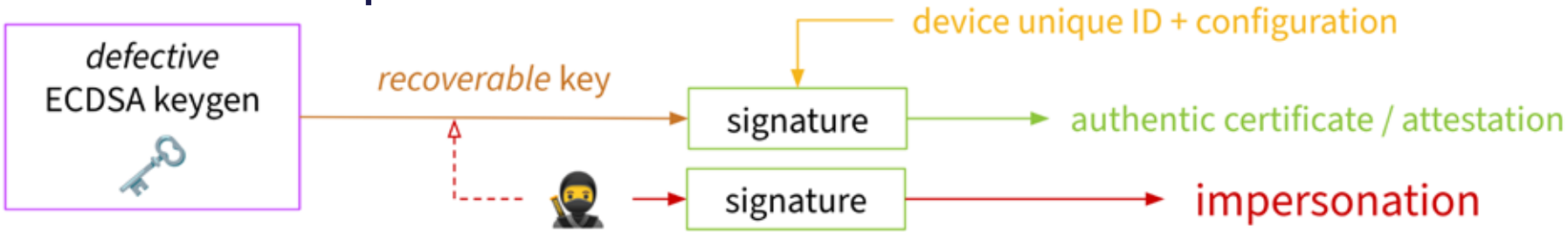
Andreas Kurth *Director of Hardware Engineering – lowRISC*

Zach Halvorsen *Software Engineer – Google*



Why focus on Design Verification in Caliptra?

- **Caliptra** serves as the **Root of Trust (RoT)** in **datacenter SoCs** (CPUs, GPUs, DPUs, TPUs, …).
- Caliptra is thus the **foundation of security** in datacenter systems.
- **Functional defects** in Caliptra **can lead to security vulnerabilities in the datacenter**. Example :

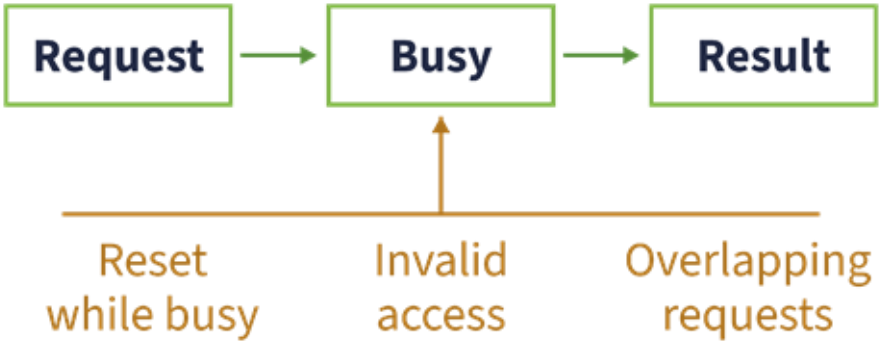


→ **Thorough Design Verification (DV) reduces the risk for all Caliptra users!**

What is DV, and why do we need it?

Design Verification (DV) checks that the hardware implements its functional and security requirements.

Normal operation can hide bugs



Rare combinations hide defects

Normal workloads explore only some combinations of states and timing.

Security must hold on error paths

Illegal requests and faults must preserve the security boundaries.

Late hardware fixes are costly

Find defects before fabrication, when the RTL can still be changed.

DV finds bugs, exposes coverage gaps and catches regressions.

Which verification gaps existed in Caliptra?

- Caliptra previously relied heavily on **software-driven tests**.
- **Coverage-driven DV** existed in closed-source environments but could **not** readily be independently **reproduced** and **reviewed**.

How to close these verification gaps?

Porting a block-level DV environment

1 Import
Start from the upstream block-level DV environment (e.g., OpenTitan).

→

2 Adapt
Swap the bus (TileLink → AHB); drop alerts and fault-injection checks.

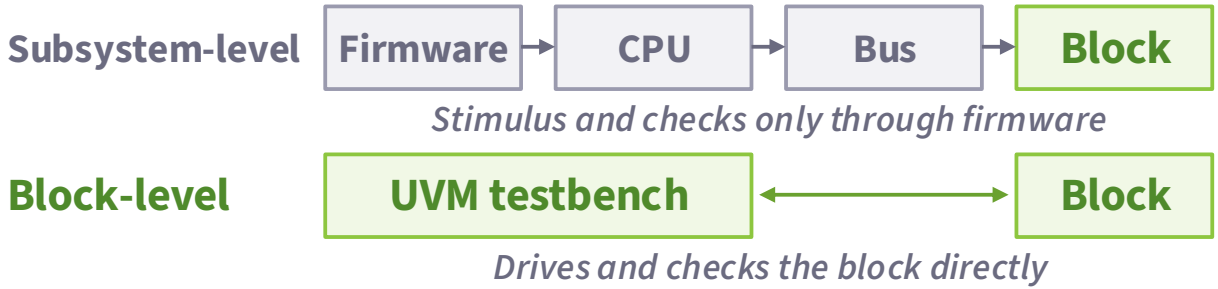
→

3 Reuse
The ported testbench verifies the features inherited from OpenTitan.

→

4 Extend
Add tests for the Caliptra-specific features added while porting.

Why test at block level?



✓ Faster, more precise stimulus

✓ More precise *checking*

✓ Random stimulus (hard with a simulated CPU)

Practical Examples of IP Verification



Example 1: VeeR (CPU)

- Open-source simulation & framework
  - Verilator (optional VCS) driven by a Python cocotb + PyUVM framework
    - ~22 block-level testbenches plus top-level PyUVM tests
    - GitHub Actions CI with line/toggle/branch/functional coverage reporting
- Open-source compliance & random verification
  - RISCV-DV for random instruction stream gen, compared against Spike/Renode; RISCOF runs the official RISC-V architectural compliance suite
- Security-focused DV
  - Temporal fault injection with variable delay skew exercises DCLS (dual-core lockstep)
  - ECC/parity error injection (ecc, icache_ecc) and memory-integrity checks

Example 2: SHA3_CTRL

- Ported KMAC environment from OpenTitan
- The OpenTitan environment needed some generalisation (cip_lib is too OpenTitan-specific)
- This resulted in nicer “common tests” (registers, interrupts, resets) than the original: changes that can be ported back again.
- After a couple of weeks’ work, all the tests run to completion and pass. This does not yet test any of the Caliptra-specific wrapper code that had been added.

Stage	Passing	Total	Pass Rate
V1	20	20	100.00 %
V2	90	90	100.00 %
V2S	25	25	100.00 %
	153	153	100.0

TOTAL	BLOCK	LINE/STATEMENT	BRANCH	TOGGLE	FSM	ASSERTION
55.98 %	89.23 %	90.28 %	78.13 %	62.71 %	47.52 %	98.21 %

Example 3: I3C

- An I3C target device implementing *OCP Secure Firmware Recovery v1.1-rc5*

- Primary DV flow is entirely FOSS (cocotb + Verilator)
  - Python-native: ~90 cocotb tests (~87k LOC), AHB + AXI configs
  - Two tiers of coverage: ~24 block/unit tests plus full-core top-level tests
  - Coverage instrumented: code coverage, functional covergroups, RTL SVA assertions, and FSM state/transition logs parsed for completeness.

- Testsuite run on every submitted change in GitHub Actions

The Power of Open Source DV



Reusable IP verification

- A **DV environment** ships **tests**, **stimulus**, **checking** and **coverage** alongside the design.
- **Integrators verify** the IP in **their own configuration** and **system context** — and **re-verify after modifications**, parameter changes or extensions.
- **Fixes** and **new tests flow back upstream**, raising confidence for everyone.

Bottom line: Open-source DV turns **verification** from a “rinse and repeat” project activity into a **reusable property of the IP**.

Reusable DV framework

- **Caliptra** and **OpenTitan** share **one DV framework**
  - **DVSim** for build, regression and coverage management;
  - **UVM agents** and **abstraction libraries**;
  - **Common IP environments**.
- This framework makes common tasks **much more efficient**
  - Running regressions
  - Merging coverage
  - Reporting results
- **Adoption** is **incremental**, not all-or-nothing.
- Because both IP and infrastructure are open, **improvements flow back upstream**

In Conclusion



Lessons learned

- **Thorough Design Verification (DV) reduces the risk for all Caliptra users.**
- **Open-source DV** turns verification from a “rinse and repeat” activity into a **reusable property of the IP.**
- **Adoption** is **incremental,** not all-or-nothing.
- A **reusable DV framework** makes common tasks much **more efficient.**
- Because both IP and framework are open, **improvements** flowing back **upstream** lead to cross-project **synergies.**

Call to Action

- Architecture and design are constantly evolving, so verification needs to be a continuous process as well. If you are a **Caliptra developer, please join**!
- Become a **Caliptra integrator** to **benefit** from this!
- Scan the QR code on the right to join the **CHIPS Alliance Caliptra project** to get involved.



Thank You!

